

Predykcja ryzyka kredytowego

Projekt z Uczenia Maszynowego

Patrycja Gromada

Przetwarzanie danych

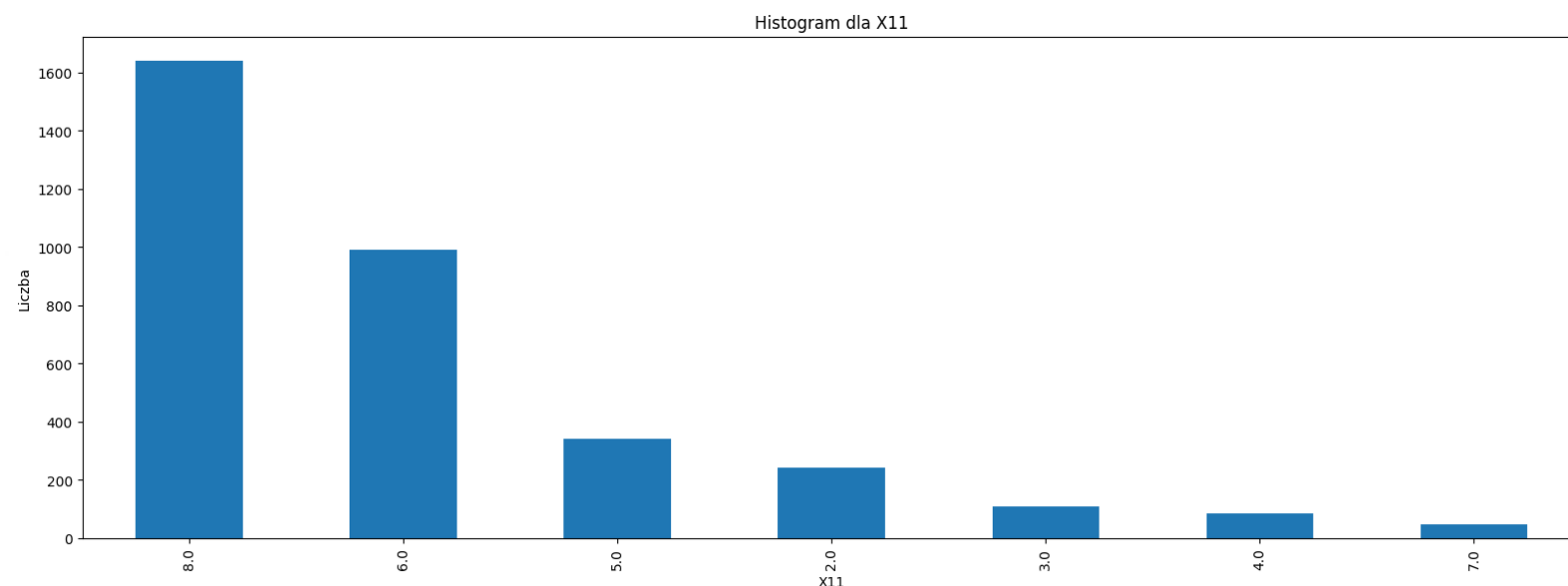
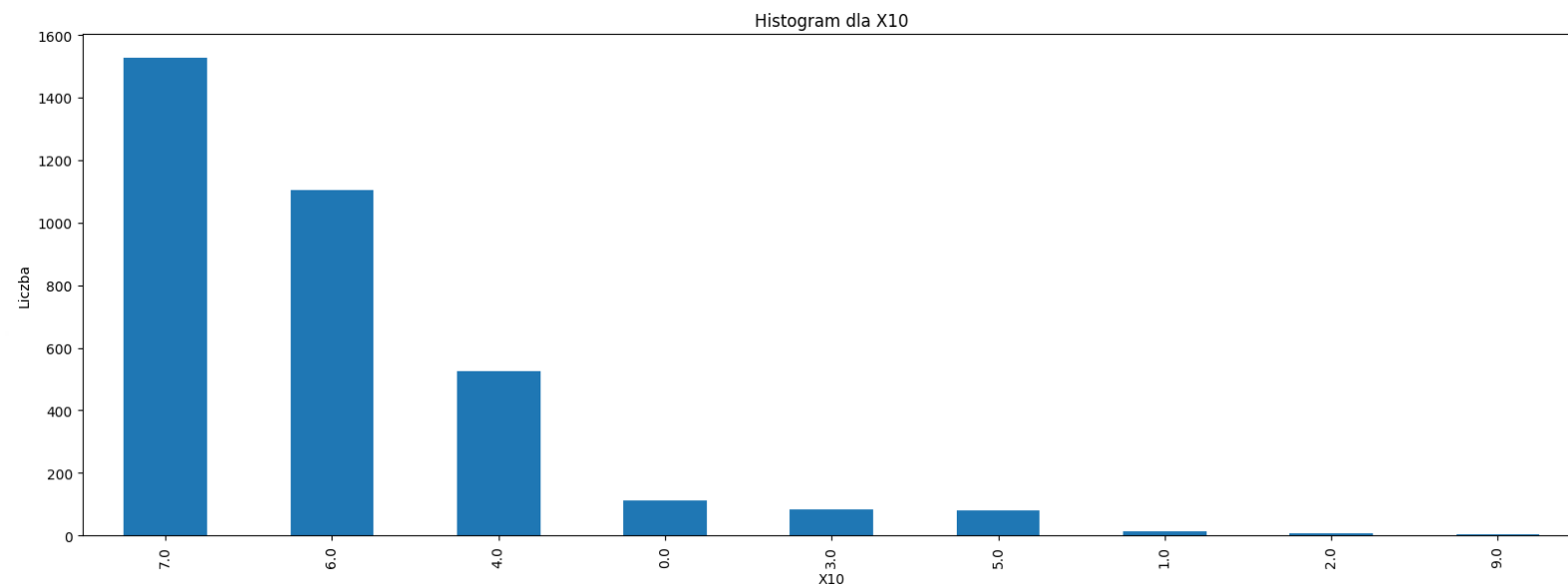
Usunięcie wierszy zawierających tylko wartość NaN.

3360x23 -> 3457x23

Skalowanie zmiennych numerycznych przez StandardScaler()

Zamiana zmiennych kategorycznych na liczby przez OneHotEncoder()

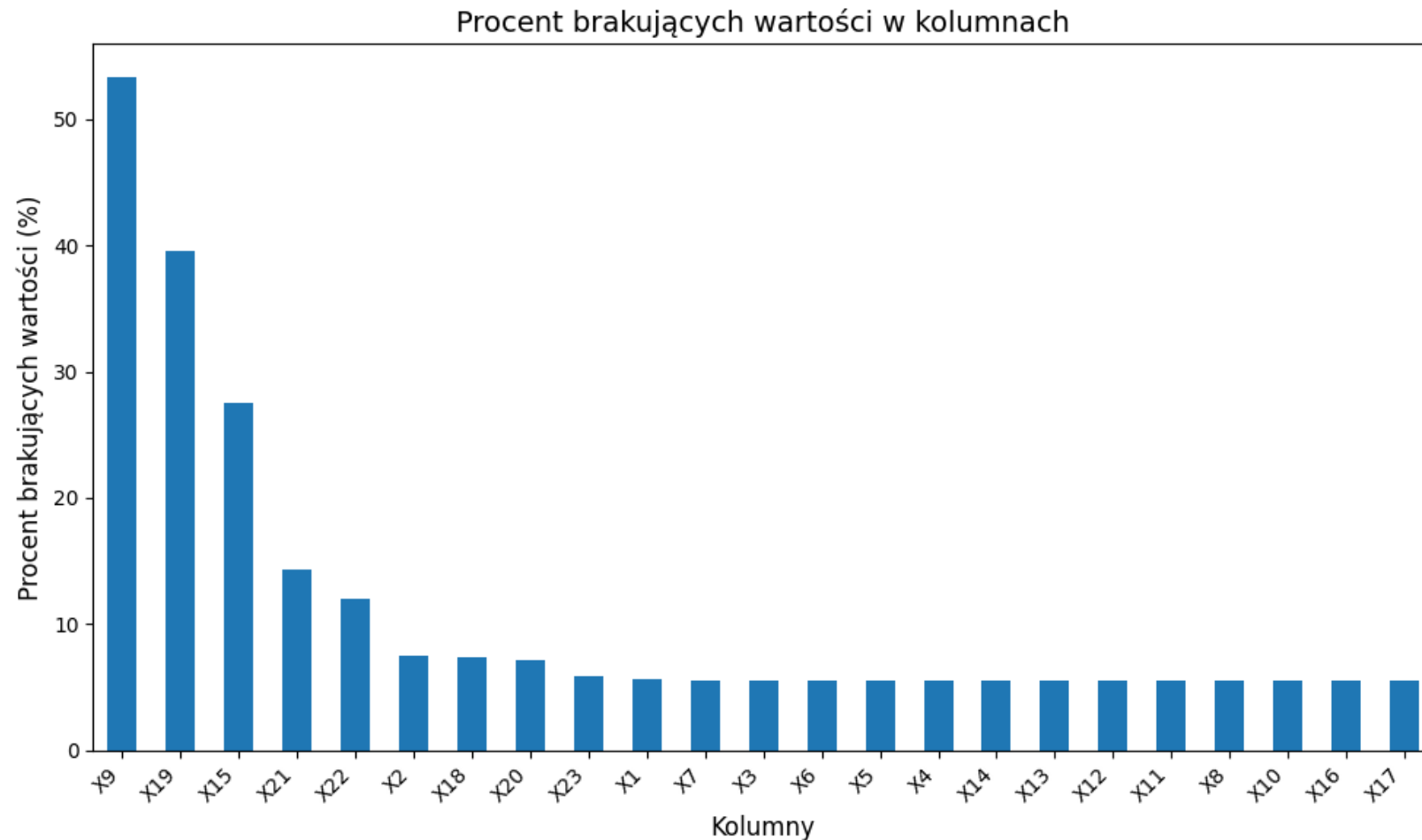
Podział zbioru danych w stosunku 8:2,
5-krotna krosvalidacja



Wykres: Histogram zmiennych kategorycznych X10 i X11

Braki danych

Uzupełnienie braków
danych poprzez KNNImputer
z 5-sąsiadami



Wykres: Procent brakujących wartości w poszczególnych kolumnach
X9 - ile miesięcy minęło od ostatniego opóźnienia w płatnościach

Modele

8 modeli 'podstawowych'

4 bagging

3 stacking

3 voting

+ domyślne modele bez wpisanych hiperparametrów

Modele (domyślne oraz z zoptymalizowanymi hiperparametrami)

Modele podstawowe:

- Decision Tree 0.7128 0.7250
Decision Tree (Default) 1.0000 0.6684
- Logistic Regression 0.7365 0.7373
Logistic Regression (Default) 0.7329 0.7455
- Random Forest 0.8176 0.7441
Random Forest (Default) 1.0000 0.7485
- KNN 0.7541 0.7186
KNN (Default) 0.7748 0.6938
- SVM 0.7585 0.7468
SVM (Default) 0.7763 0.7412
- Extra Tree 0.9141 0.7367
Extra Tree (Default) 1.0000 0.7440
- Gradient Boosting 0.7515 0.7457
Gradient Boosting (Default) 0.7918 0.7458
- AdaBoost 0.7391 0.7657
AdaBoost (Default) 0.7522 0.7601

Bagging:

- Decision Tree (Bagging) 0.7128 0.7250
Decision Tree (Bagging Default) 1.0000 0.7556
- Logistic Regression (Bagging) 0.7365 0.7373
Logistic Regression (Bagging Default) 0.7336 0.7455
- KNN (Bagging) 0.7558 0.7288
KNN (Bagging Default) 0.7856 0.7096
- SVM (Bagging) 0.7621 0.7485
SVM (Bagging Default) 0.7824 0.7384

Komitety:

Stacking (DT, LR, RF, KNN, SVM) 0.7447 0.7385
Stacking (ExtraTree, GradientBoosting, AdaBoost) 0.7756 0.7485
Stacking (Bagging) 0.7455 0.7401
Voting (DT, LR, RF, KNN) 0.7669 0.7415
Voting (ExtraTree, GradientBoosting, AdaBoost) 0.8082 0.7456
Voting (DT, LR, RF, KNN, Bagging) 0.7631 0.74284

Voting i Stacking (z zoptymalizowanymi hiperparametrami)

Kombinacja 1: Drzewo Decyzyjne,
Regresja Logistyczna, Las Losowy, k-NN.

Kombinacja 2: ExtraTree,
GradientBoosting, AdaBoost.

Kombinacja 3: Drzewo Decyzyjne,
Regresja Logistyczna, Las Losowy, k-NN,
SVM, Bagging - Drzewo Decyzyjne,
Bagging - Regresja Logistyczna, Bagging
- Las Losowy, Bagging - k-NN.

Model Voting	BA (Train)	BA (Test)
Kombinacja 1	0.7669	0.7415
Kombinacja 2	0.8082	0.7456
Kombinacja 3	0.7631	0.7428

Model Stacking	BA (Train)	BA (Test)
Kombinacja 1	0.7447	0.7385
Kombinacja 2	0.7756	0.7485
Kombinacja 3	0.7455	0.7401

Bagging - wpływ

Interesującym aspektem może być zmiana wartości metryki w zależności od zastosowania baggingu:

Model	BA Test (Bez Baggingu)	BA Test (Z Baggingiem)
Drzewo Decyzyjne	0.7250	0.7250
Regresja Logistyczna	0.7373	0.7455
k-NN	0.7186	0.7096
SVM	0.7486	0.7384

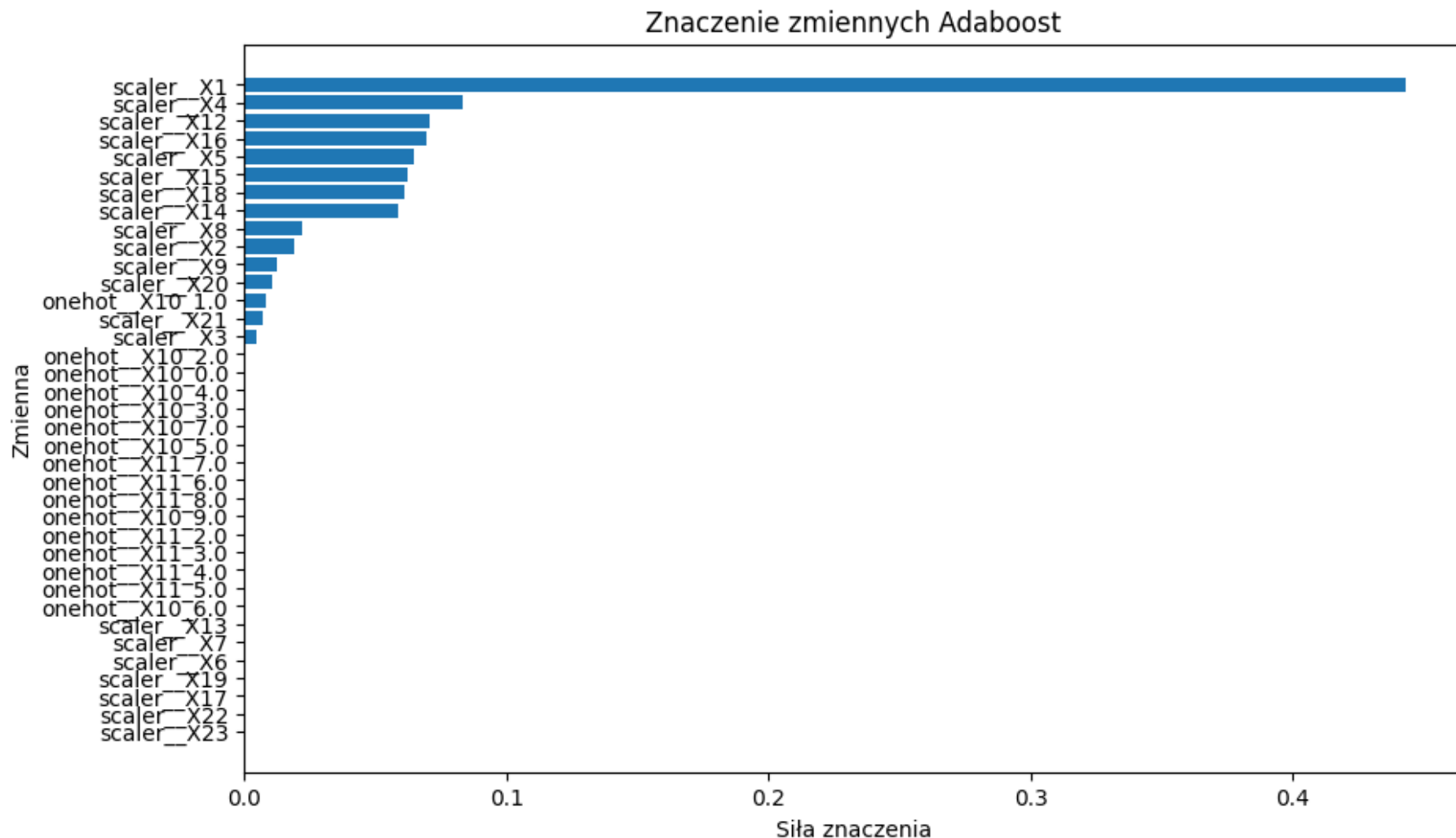
To już prawie koniec...

Po posortowaniu wszystkich wyników według kolumny BA (Test), pięć najlepszych modeli to:

Model	BA (Trening)	BA (Test)
AdaBoost	0.7391	0.7657
AdaBoost (Domyślny)	0.7522	0.7601
Bagging - Drzewo Decyzyjne (Domyślny)	1.0000	0.7556
Stacking (Kombinacja 2)	0.7756	0.7485
Bagging - SVM	0.7621	0.7485

Ważność zmiennych w AdaBoost

1. X1 – Zbiorcza ocena wskaźników ryzyka
2. X4 - Średnia liczba miesięcy, od kiedy istnieją wszystkie konta.
3. X12 - Liczba wszystkich kont kredytowych.
4. X16 - Liczba zapytań o kredyt w ostatnich 6 miesiącach
5. X5 - Liczba kont z pozytywną historią spłat



Ostatecznie...

Na podstawie poprzedniej tabelki przetestowałam predykcje 3 najlepszych modeli w aplikacji. Poniżej znajdują się otrzymane wyniki:

Model	BA
AdaBoost	0.7431
Bagging - Drzewo Decyzyjne (Domyślne)	0.7503
Stacking (Kombinacja 2)	0.7559



Stacking (Kombinacja 2)

tworzony na modelach:

ExtraTree

- 'bootstrap': True, 'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 100.,

Gradient Boosting

- 'max_depth': 2, 'min_samples_leaf': 6, 'min_samples_split': 2, 'n_estimators': 50,

AdaBoost

- 'algorithm': 'SAMME', 'estimator':

```
DecisionTreeClassifier(max_depth=1),  
    'n_estimators': 200
```

Dziękuję za uwagę

Patrycja Gromada